

Smart AI based model:CNNs for vechicle detection,reinforcement learning for traffic light control

```
import random

import numpy as np

import time

class SmartTrafficController:

    def __init__(self, num_directions=4):

        self.num_directions = num_directions

        self.q_table = np.zeros((num_directions, 2)) # 2 actions: 0 – Red, 1 – Green

        self.vehicle_counts = np.zeros(num_directions, dtype=int)

        self.learning_rate = 0.1

        self.gamma = 0.9 # discount factor

    def detect_vehicles(self):

        """Simulate CNN vehicle detection (replace with real OpenCV/TensorFlow model)."""

        self.vehicle_counts = np.random.randint(0, 20, size=self.num_directions)

        print(f"Detected vehicles: {self.vehicle_counts}")
```

```
def select_best_direction(self):
```

```
    """Select direction with most vehicles for green signal."""
```

```
    return np.argmax(self.vehicle_counts)
```

```
def apply_traffic_signal(self, green_direction):
```

```
    """Apply green signal to one direction, red to others."""
```

```
    print(f"\n 🚦 Green light for direction {green_direction}")
```

```
    for i in range(self.num_directions):
```

```
        light = "GREEN" if i == green_direction else "RED"
```

```
        print(f"Direction {i}: {light}")
```

```
def update_q_table(self, direction, reward):
```

```
    """Update Q-values based on reward."""
```

```
    old_value = self.q_table[direction][1]
```

```
    self.q_table[direction][1] += self.learning_rate * (reward + self.gamma *  
np.max(self.q_table[direction]) - old_value)
```

```
    print(f"Updated Q-value for direction {direction}:  
{self.q_table[direction][1]:.2f}")
```

```
def calculate_reward(self, direction):
```

```
    """Reward based on traffic reduction; fewer vehicles = better."""
```

```
    vehicles = self.vehicle_counts[direction]
```

```
    return 1 if vehicles < 10 else -1
```

```
def run_simulation(self, cycles=10):
```

```

for cycle in range(cycles):

    print(f"\n===== Cycle {cycle + 1} =====")

    self.detect_vehicles()

    best_direction = self.select_best_direction()

    self.apply_traffic_signal(best_direction)

    reward = self.calculate_reward(best_direction)

    self.update_q_table(best_direction, reward)

    time.sleep(1)

if __name__ == "__main__":

    controller = SmartTrafficController(num_directions=4)

    controller.run_simulation(cycles=5)

```

Output:

```
===== Cycle 1 =====
```

```
Detected vehicles: [8 5 1 8]
```

 Green light for direction 0

```
Direction 0: GREEN
```

```
Direction 1: RED
```

```
Direction 2: RED
```

Direction 3: RED

Updated Q-value for direction 0: 0.10

===== Cycle 2 =====

Detected vehicles: [5 4 15 0]

 Green light for direction 2

Direction 0: RED

Direction 1: RED

Direction 2: GREEN

Direction 3: RED

Updated Q-value for direction 2: -0.10

===== Cycle 3 =====

Detected vehicles: [5 7 19 2]

 Green light for direction 2

Direction 0: RED

Direction 1: RED

Direction 2: GREEN

Direction 3: RED

Updated Q-value for direction 2: -0.19

===== Cycle 4 =====

Detected vehicles: [1 18 12 5]

 Green light for direction 1

Direction 0: RED

Direction 1: GREEN

Direction 2: RED

Direction 3: RED

Updated Q-value for direction 1: -0.10

===== Cycle 5 =====

Detected vehicles: [18 7 17 0]

 Green light for direction 0

Direction 0: GREEN

Direction 1: RED

Direction 2: RED

Direction 3: RED

Updated Q-value for direction 0: -0.00

[Colab paid products](#) – [Cancel contracts here](#)

5scompleted at 22:27